

place every instance of registry.gitlab.com/gitlab-org/build/cng/gitlab-webservice-ee:v15.5.1
with the new image
kubectl -n <gitlab-namespace> edit deployment <webservice-deployment>

To revert the patch, you can edit the deployment to use the original image.

Note:

- This process only applies to the Rails application (the GitLab repository).
You will need to patch a different image depending on the GitLab component, you want to patch.
- This is different with Patching the Rails code in the toolbox pod. Patching rails code directly in the toolbox pod will not apply the patch to the rails code that is serving the requests to the users.
- You will need to create a new image with the patch if you upgraded the helm chart to a newer version of GitLab.

SECTION: support/workflows/personal_data_access_account_deletion.md

Purpose

This document provides an overview of what account deletion and data access requests are and who is responsible for processing the different aspects of each request. To learn how to process each type of request as a Support Engineer, refer to the workflows page.

Under numerous global and national data privacy laws, users can request to have their GitLab SaaS accounts, along with any other information that we have stored about them, deleted. They can also request to obtain more details about their data, including information about what data GitLab has stored about them.

Users requesting data deletion are required to confirm their intent to delete at the time of their request submission. For GitLab SaaS and full deletion requests only, form entries for username, email, and paid namespace memberships will be automatically validated. All other requests require manual review and validation (where applicable) until further improvements have been implemented.

Scope

This process applies to personal data requests submitted by both personal users or corporate GitLab SaaS subscription holders and customers. Account deletion requests for corporate subscription holders require validation and approval by the paid namespace account holder with a current GitLab subscription contract. As part of this approval process, the account holder of the paid namespace (owner) must remove the subscription holder (user account) from their paid namespace.

Roles and Responsibilities

| Role | Responsibility|

| ---- | ----- |

| Support Team | Maintaining this deletion process handbook page and related deletion request project and issue templates |

| Support and Legal Team | Responsible for approving significant changes to this standard |

| Support and Legal Team | Responsible for approving exceptions to this standard |

| Support Global Readiness | Responsible for resourcing, namely that there are enough Support Engineers to complete requests in a timely manner |

| GitLab system owners | Processing user deletion for each system as required by the data deletion issue created through this process |

Streamlining Your Workflow

Every request will go through multiple stages, denoted by labels, before they're closed out as completed. Keeping track of which stage of the process each request assigned to you is at is essential to processing them efficiently. Use one of the following two methods to do this:

1. Create a new issue board in the Personal Account Requests Service Desk issue tracker that is scoped to only show issues that you are assigned to. Then, create new lists for each label used in the process. Reference this board for an example.

1. Use the already existing Issue Tracking Board and filter it so that only issues that are assigned to you are displayed by typing in Assignee = yourusername in the search field.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Controlled Document Procedure

SECTION: support/workflows/pii_removal_requests.md

Overview

Personal Data (also known as personally identifiable information) is any data, individually or when combined with other data, that identifies, relates to, describes or is reasonably capable of being associated with or linked to an identifiable natural person, whether directly or indirectly. Personal Data can be sensitive or non-sensitive. Non-sensitive Personal Data is information such as a public avatar, name, email addresses, company/organization name, job title, country, social media handles and biographic information as well as public user profile information. Sensitive Personal Data is information which, when disclosed, could result in harm to the individual whose privacy has been breached, such as government identification numbers, genetic data, health information, religious affiliation, political affiliation, or photos.

Although GitLab does not request or intentionally collect any Sensitive Personal Data, if a user stores such data in a publicly accessible repository, the data can be publicly accessible via a GitLab.com. While this list is not exhaustive, these are examples of the different types

of Personal Data which could be made available intentionally or non-intentionally.

Note that a Personal Data removal request is intended for the removal of personal data that can or is being used to identify or link to a specific individual for potentially malicious intent. This type of removal request is not to be used to exercise an individual's right to deletion under any global data privacy laws.

Keywords: Personal Data, Name, Social Security Number, Date and Place of birth, Mother's Maiden Name, Biometric Record, Résumé, Education Information etc.

Personal Data removal requests can be sent to <abuse@gitlab.com>.

Workflow

1. Vet the request.
1. Verify that the reported links to the content is present in the request?
1. Is the content still live?
1. If the content is no longer available. Notify the requester and close the request. (cc <abuse-reports@gitlab.com>)
1. If the content is still live, forward the request to Abuse for further review.
1. The Abuse Team will comment on the ticket advising on the next steps or steps taken.

It is important for the Abuse team to review the request before forwarding it to the reported account. The Abuse Team will create a new ticket and forward the initial notice, including the reference ticket in Internal Notes. Abuse Team will take over from here.

SECTION: support/workflows/printing_ticket_thread_without_internal_notes.md

Printing a clean ticket thread

Zendesk does not have the ability to print ticket threads without the internal notes. Even if you filter by only public conversations and click the Print ticket action, the generated output will still contain internal comments.

Sometimes there is a need to print ticket threads to share the content with the customers or for documentation purposes.

An easy way of cleaning up the HTML DOM output is by using a javascript bookmarklet to do it.

1. Click on your bookmark bar in your browser and select Add page...
1. Give it a name of Remove Zendesk Internal Notes
1. Paste the following code snippet into the URL input box

javascript

```
javascript:(function(){null!==document.getElementById("ccs")&&document.getElementById("ccs").remove(),null!==document.getElementById("fields")&&document.getElementById("fields").remove(),null!==document.getElementById("customfields")&&document.getElementById("customfields").remove();const list=document.querySelectorAll("comments > .comment");for(let item of list){let e=item.chi
```

```
ldren,l=[]&#x2D;filter.call(e,e=>[].includes.call(e.classList,"mast"))[0],t=l.children,s=[]&#x2D;filter.c
all(t,e=>[].includes.call(e.classList,"internalnote"))[0];void 0!==s&&item.remove())})();
```

To use the bookmarklet, go to any ticket and click Print ticket. That will bring you to a new page with the print dialog opened. Cancel the print dialog and click the Remove Zendesk Internal Notes bookmark to remove the internal notes. Please verify that the content you want removed has been actually removed. You can then print to paper or save as a PDF

Formatted bookmarklet with comments

```
javascript
javascript:(function(){
  // Remove the cc
  if (document.getElementById('ccs') !== null) {
    document.getElementById('ccs').remove();
  }
  // Remove the fields section
  if (document.getElementById('fields') !== null) {
    document.getElementById('fields').remove();
  }
  // Remove the custom fields section
  if (document.getElementById('customfields') !== null) {
    document.getElementById('customfields').remove();
  }
  // Select all comments
  const list = document.querySelectorAll('comments > .comment');
  for (let item of list) {
    // Select the children elements of the comments
    const commentChildren = item.children;
    // Pull out the mast element specifically
    const mastElement = [].filter.call(
      commentChildren, element => [].includes.call(element.classList, 'mast')
    )[0];

    // Select the children elements of the mast element
    const mastChildren = mastElement.children;

    // Pull out the internalnote element
    const internalNoteElement = [].filter.call(
      mastChildren, element => [].includes.call(element.classList, 'internalnote')
    )[0];

    // If the internal note element exists, we remove the main comment
    if (typeof internalNoteElement !== 'undefined') {
      item.remove();
    }
  }
})();
```

SECTION: support/workflows/quick-reference.md

{{% include "includes/support-quick-reference.md" %}}

SECTION: support/workflows/rbl_delisting.md

Overview

In some rare cases, you may need to reach out to Mailgun support to remove our IP addresses from a Real-time Blocklist (RBL). This workflow will cover how to verify and resolve these issues.

Check for Suppression in Mailgun

If a customer is reporting that they are not receiving a confirmation email, you'll want to go through the confirmation email workflow first. When in the Mailgun panel, check for a suppression.

Verify the RBL listing

In the logs, if you see a mention of a rejection due to abuse or explicit confirmation of a blocklist, proceed to the link and look up the IP address. Every provider will be different, but if a lookup database is available, you should verify that it is listed and if a reason is stated.

text

```
"delivery-status": {  
  "tls": true,  
  "mx-host": "mx1.external-mailserver.com",  
  "attempt-no": 1,  
  "description": "",  
  "session-seconds": 1.7985761165618896,  
  "code": 550,  
  "message": "5.7.1 H:M11 [192.237.158.143] Connection refused due to abuse. Please see  
https://mailspike.org/iplookup.html or contact your E-mail provider."}
```

You can confirm that the IP address is ours by doing a "host" lookup on the IP address. It should always end in mg.gitlab.com.

text

```
> host 192.237.158.143  
143.158.237.192.in-addr.arpa domain name pointer do158-143.mg.gitlab.com.
```

Open a Ticket With Mailgun

Navigate to Mailgun > Support and click "Create Ticket". Be sure to include:

1. The IP Address that is blocklisted
1. The name of the RBL and link to it (Found in the Message returned by the mail server)
1. Copy of the Logs

Follow up

Be sure to follow up with the customer and let them know that we've requested support through Mailgun. They should return a response within 24 hours. If we are blocklisted due to a specific account, please check in with Site Reliability Engineers in production.

SECTION: support/workflows/reinstating-blocked-accounts.md

This workflow page will describe how to action on Locked, Blocked and Banned accounts. Sometimes users believe they are blocked, but their accounts are locked. There are several ways to verify:

1. The best way to view this information is via the Zendesk User Lookup app (part of the GitLab Super App), through the Locked and State fields.
1. The Admin User UI in /admin/user/USERNAME will say (Locked), (Blocked) or (Banned) next to the name at the top.
1. The Users API through the URL <https://gitlab.com/api/v4/users/<userid>> in your browser while logged in as an Admin User, also indicates the locked and state status of the user.

Our implementation of Arkose Protect does not affect account locking, but instead can prevent users from signing in without solving the challenge.

Locked accounts

When a user has been identified as locked, you can use the Support::SaaS::Account Locked macro to help explain the situation to the user. A user can be locked if:

2FA is not enabled:

- They do not have 2FA enabled and there have been 3 or more failed login attempts within 24 hours.
- Accounts are not unlocked automatically. The user receives an email with a six-digit unlock code after a successful login. They are then redirected to a verification page where they can unlock their account by entering the code. The code is valid for 60 minutes only, but they can request a new code by clicking a link on the verification page.

2FA is enabled:

- There have been 5 or more failed login attempts within 10 minutes.
- Accounts are unlocked automatically after a 10 minute waiting period.

If the user does not receive a verification email with the 6-digit code, it's likely that the

primary email address is inactive or inaccessible. If a user does not have access to their primary email address, they cannot unlock their account or reset their password. Consider other workflows such as swapping email addresses if a user is not able to access their primary email.

All verification emails with unlock codes and password reset emails bypass Mailgun suppressions. Mail delivery of these emails can also be seen in Mailgun.

You can see Account Locked states in Kibana by searching for `json.message: Account Locked`. Here's an example of what it might look like in Kibana:

Manual unlock

A user should attempt all self-serve methods first.

If self-serve methods have been exhausted and a member of a group with a paid subscription still cannot access their account, we can consider a manual unlock if necessary. For example, if a user cannot receive external email to receive codes, a manual unlock may be required. Note that only an admin user can modify a user account on SaaS.

Process:

1. Follow the locked accounts workflow above and ensure that the user has exhausted all self-serve methods first.
1. For other cases, comment or create an issue as applicable.
1. Do an account ownership verification.
1. Unlock the account from the admin area
1. Add an admin note.

Feature request for group owners to self-serve is in [anti-abuse339](#).

Change risk assessment (Credit Card verification)

If a user has failed credit card verification or cannot use a credit card, follow the below process to verify the user in order to change their risk level. Please see [this issue](#) and the documentation for more details.

Note: This process can only be done for free users if it's determined that they are impacted by a GitLab bug, such as [customers3811](#).

Process:

1. Follow the steps above for manual unlock.
1. While in the admin area for the user, scroll to the Custom Attributes section.
1. Change the field for `arkoseriskband` from high to medium.
1. If necessary, unlock the account.
1. Add an admin note.
1. Click Save when done.

Identity verification exemption requests

Accounts unable to verify with a credit card and/or phone number

When a user is unable to complete credit card and/or phone number verification, an exemption can be requested from the Trust and Safety team. This is also sometimes referred to as "identity verification".

Details of the process and instructions are available in the Internal Handbook.

Blocked Accounts

This workflow is used to determine if a blocked or a banned user can be reinstated. All blocked accounts should have an admin note with a link to a relevant issue.

Why is account blocked?

If the account is blocked, look for the admin note on the account to determine why it has been blocked.

- The GitLab user lookup app in Zendesk will show the admin notes for the user if they have contacted support using the email address associated with their account. Alternatively -
- If you have access to ChatOps you can use the below command in any chatops enabled Slack channel to read admin notes for the user
> /chatops run user find <username or email>

User initiated self-serve deletion

If the Admin Note is User deleted own account on {timestamp}, this means the user initiated the self-serve deletion. See Cancelling delayed account deletion.

1. Free user accounts need to wait 7 days, starting the day of the deletion request, to create a new account with the same email address or username. Use the `Support::SaaS::Gitlab.com::Blocked Accounts::Blocked due to account deletion` macro.

1. Exceptions in which Support can force deletion - If a user is not part of a paid namespace but needs to be added to a paid namespace (user or top-level group owner creates the ticket), or if a paid user added below the top-level group and still subject to the 7-day delay period (True until this issue is fixed): then Support can follow the steps below to bypass the 7 day delay:

- Use the `Support::SaaS::Gitlab.com::Blocked Accounts::Blocked due to account deletion` macro and ask for explicit permission from the user to bypass the 7d wait period and delete the account.

- When confirmation is received, SE (with Admin access) deletes the account.

- SE updates the ticket with the result of the deletion.

1. Paid user accounts that are direct members of a top-level paid namespace have no deletion delay and their account is deleted immediately (within 1 hour), see this MR.

Embargoed countries

If the block or complaint is related to access from an embargoed country, use the `Support::SaaS::Gitlab.com::Abuse::TOS Section 10 (Embargoed Countries).md?reftype=heads` macro.

- If the user provides the requested information, then complete the Trust and Safety Account Reinstatement Request template in the Trust and Safety Operations tracker. Otherwise, reaffirm the block cannot be removed.
- Proceed with this action for both free and paid users.

Business and regulatory obligations (China region)

Users may be blocked in order to comply with business and regulatory obligations in Mainland China, Hong Kong and Macau. This will be reflected in an admin note on the user's account.

More information and the support workflow for these accounts can be found in the Internal Handbook.

Professional Services migrations

Professional Services migrations can also block users as part of their process. Admin notes for migrations were added as of 2022-08-19 through this issue. Older migrated accounts may not have an admin note. As of 2024-09-18, requests to unblock accounts that were blocked during a Professional Services migration are worked automatically (see STM 6336).

If you receive a ticket for an unblock request and you think it should have been automatically worked, you should:

1. Check that all ticket fields are at expected values
1. Notify Support Operations so that they can investigate
1. Work the ticket manually using the guidance below to ensure a timely resolution for the requestor

If a ticket was not automatically worked, Support can manually unblock the user in the following cases:

- Blocked users or top-level group owners can submit a support ticket to be unblocked. Once they are verified, the user can be unblocked. Leave an admin note on the user stating they were unblocked, with the date and ticket number.
- For Enterprise users, the owner of the top-level namespace the user belongs to can submit the ticket. Follow the account verification, and add an admin note as usual, including if it was user or owner requested.
- You can also ask for clarification or assistance in the `professionalservices` channel if needed.
- Proceed with this action for both free and paid users.

No admin note and none of the above

For all other cases, including no admin notes that are not a part of PS migrations, complete the Account Reinstatement Request template in the Trust and Safety Operations tracker. A security member of the team will review the request within 24 hours. If the request is urgent, please reach out in the abuse Slack channel.

1. Send the `Support::SaaS::Gitlab.com::Blocked Accounts::Escalated-TrustAndSafety` macro for the initial response to the user.

1. If established it is not a Trust and Safety block, or is blocked as a result of a SM-to-SaaS migration (conducted with or without Professional Services) AND they are verified, then:
 - Paid accounts can be unblocked with authorization from a user with the Owner role in the top level namespace.
 - Free accounts can only be unblocked under exceptional circumstances and in combination with leadership approval.

Account is successfully unblocked

If account is unblocked, use the Support::SaaS::Gitlab.com::Blocked Accounts::Account Reinstated- Success macro to notify the user the account has been unblocked.

Account to remain blocked (no reinstatement)

If the final decision from Trust and Safety is that the user's account is not to be reinstated, apply the Support::SaaS::Gitlab.com::Blocked Accounts::RemainBlocked macro. This will provide a standard statement to the user and the ticket will be closed.

Applying the macro will provide the user no opportunity of reply through the existing ticket, care should be taken before applying the macro to ensure that this is the intended action.

Banned accounts

1. Proceed with the next steps only if at least one of the following scenarios is true:
 1. The email address the user has used to raise their request matches an email address associated with the account the request is intended for. This criteria can be applied to both free users and customers.
 1. The user account is classified as an Enterprise user and an owner of the top-level group raises the ticket.
1. Complete the Trust and Safety Account Reinstatement Request template in the Trust and Safety Operations tracker. A security member of the team will review the request within 24 hours. If the request is urgent, please reach out in the abuse Slack channel.
1. Send the Support::SaaS::Gitlab.com::Blocked Accounts::Escalated-TrustAndSafety macro for the initial response to the user.
1. If account is restored, use the Support::SaaS::Gitlab.com::Blocked Accounts::Account Reinstated- Success macro to notify the user the account has been restored. Otherwise, provide the reasoning from the Reinstatement Request as to why their account will remain banned.

Cancelling delayed account deletion

It is possible for self-initiated account deletion to be cancelled within the 7-day delay period. See Unblocking the account will cancel the account deletion.

A request to cancel the deletion of an account may be made by a member of a paid group or a top-level owner if the user is an Enterprise user. We do not cancel account deletion for free users.

Process:

1. The paid user or top-level owner must successfully pass account verification.

1. Unblock the user and leave an admin note on the user stating they were unblocked, with the date and ticket number.

NOTE:

The rest of this page is for reference only and should be updated to point to Security's workflow.

Policy Reference

1. All decisions about account reinstatement are final and there is no process for appeals.
1. These criteria are to be taken as examples only, and not as binding principles.
1. If the account violates our ToS again within a 12 month period, it could result in being permanently banned.

An account can be reinstated when

1. The user agrees to remove the content in question within the requested time frame.
1. The user has provided a sufficient use case for violating our Terms of Use.
1. The user agrees to remove or export the content away from GitLab.com within 24 hours.
1. The DMCA/Trademark complaint has been resolved.

An account cannot be reinstated when

1. The user refuses to take corrective measures on the account.
1. The user continues to violate our ToS.
1. The user intentionally violates our ToS.
1. Any abusive language or hostile activity towards the support engineer.
1. The account presents damage to the GitLab business and/or brand.

SECTION: support/workflows/remove_validation.md

Overview

This workflow covers how to remove validation for a GitLab.com user account. All free users created after May 17, 2021 require a credit or debit card validation to use shared runners on GitLab.com.

> Credit card validation is not connected to a customers.gitlab.com account or Zuora subscription.

Workflow

You must ensure the customer is aware that removal of validation will result in their GitLab.com account being unable to use shared runners until they provide a credit or debit card for validation.

1. Complete the account ownership verification workflow to verify the account owner and that you have received their permission to proceed. Please use the standard data classification definition. The peer review is not mandatory for these cases.

1. Sign into your admin account and locate the customer's user account.

1. Under the account tab, click Edit.

1. Uncheck the "Validate user account" box.

1. Add an Admin Note.

1. Save your changes.

1. Send the following message to the user:

text

Hi,

We have now processed this request. Your credit card validation has been removed and your credit card details have been deleted from your GitLab.com account.

Regards,

SECTION: support/workflows/removing-sensitive-information.md

Please see [Handling Sensitive Information With GitLab Support and Removing Information From Tickets](#)

SECTION: support/workflows/residency_doc_request.md

Overview

On occasion customers may request a Certificate of Fiscal Residence, follow the process below to handle these requests.

Process

1. Refer the customer to the GitLab Finance handbook where they can see when the form will be available.

1. If the customer needs a copy they can email their account representative directly or send an email to <tax@gitlab.com>.

SECTION: support/workflows/restore_requests.md

Overview

Use this workflow when a customer requests that we restore data from GitLab SaaS that has been deleted. Note that user accounts and contributions can never be restored if deleted. This workflow is specifically for handling deletions of projects and groups. It is important to note

that container images stored within deleted groups or projects cannot be recovered.

Process

1. Check whether the customer has special provisions in their contract that might affect the processing of this request. A note of these provisions in the contract are located in the notes section of the organization in Zendesk. Look at the organization information, or the internal note at the beginning of the ticket.

- >Note: The best way to maintain this information is being discussed in this issue.

1. Ensure that the request meets the necessary criteria for restoration.

If the criteria for restoration is not met:

1. Reply to the ticket with the Support::SaaS::Gitlab.com::Restore requests::Does not meet restore criteria macro.

For further guidance in these cases, please see the internal handbook.

If the criteria for restoration is met:

1. Reply to the ticket with the Support::SaaS::Restore requests::Meets restore criteria macro, which informs the customer of the limitations of restoration and asks them to confirm that they want us to proceed.

1. Wait for the customer to reply and confirm.

1. Once you've received confirmation open an Infrastructure Issue using the request-gitlab-com template. Along with any information requested in the template be sure to include:

- A Summary of the situation.
- A link to group or project.
- A link to ticket.
- A link to bug (if applicable).

1. Inform the customer that the restoration order has been put in and is in progress.

SECTION: support/workflows/restore_requests_dedicated.md

Overview

Use this workflow when a customer requests that we restore projects or groups that have been deleted from a GitLab Dedicated instance. This workflow is specifically for handling deletions of projects and groups.

Process

1. Work with the customer to explore the self-serve options for recovering this data.

Specifically:

1. Have the customer view project pending deletion. If delayed project deletion is enabled, they may be able to Restore the deleted project(s).

1. Have the customer check whether having one of their team members git push a local clone

would suffice, clarifying that project issues, merge requests, settings etc. won't be recovered.

1. If the customer can not self-serve the recovery, inform them that we will not be able to assist further with the request

If you are unsure how to communicate this news to the customer, ask in supportgitlab-dedicated or reach out to the customer's CSM for guidance.

The Disaster recovery feature in GitLab Dedicated is intended for complete disaster recovery scenarios.

Additional Resources

- Restoring Deleted Data Requests for GitLab.com
- Updates to GitLab's delayed deletion feature for projects and groups

SECTION: support/workflows/saas_feature_flags.md

Overview

This workflow covers cases where a customer requests support to enable feature flags on their GitLab.com groups, projects, or users.

Enabling feature flags can be done via ChatOps. Before you can use ChatOps, you will need to request access.

Process

Once the request has been received via a support ticket:

1. If the feature flag issue does not already say it can be enabled for customers, comment on the feature flag issue to confirm that the product and development teams are comfortable with Support turning on the feature flag for customers.
1. Add the customer's Account Owner/Customer Success Manager found in Zendesk as a CC on the ticket so that they are aware of the request.
1. Run the appropriate ChatOps command to enable it.
1. Once enabled, add an internal comment on the feature flag issue with:
 - A screenshot of the chatops response on the feature flag issue. This should include the group, project, or user that the feature flag is scoped to.
 - The relevant zendesk ticket.

ChatOps

To enable a feature flag using ChatOps, follow the process described in the Feature Flags documentation.

Typically you will want to set the Feature Flag by actor and run one of the following in the

production Slack channel. Be sure to replace the placeholders.

- For project-actor: `/chatops run feature set --project=<path/to/project1,path/to/project2> featureflagtobeenabled true`
- For group-actor: `/chatops run feature set --group=<groupnamespace> featureflagtobeenabled true`
- For user-actor: `/chatops run feature set --user=<username> featureflagtobeenabled true`

If you are unsure how it should be applied, reach out to the appropriate owner or team on the Feature Flag issue. You can also test against your own group, project, or user but should ensure to disable the feature flag once you are done testing.

SECTION: support/workflows/saas_sm_cheatsheet.md

Overview

The purpose of this page is to provide a single reference point that Support Engineers can use when they need troubleshooting tips for various problems on any GitLab platform. The goal is to make it easier for each Support Engineer to resolve support tickets regardless of the GitLab platform involved.

How to use this page

Find the topic of interest to you, and follow the instructions there in the table row that matches the appropriate GitLab platform: Self Managed, SaaS, or Dedicated.

Logs and Performance

:----	:	----

[Self-managed] GitlabSOS is a great way to gather all logs from a customer instance in order to investigate further on potential issues.
Similar to GitLabSOS, use KubeSOS if your customer uses Kubernetes instead of source or omnibus installations.
Fast-stats is a great way to compare performance analytics from and between GitLab Logs.
Green-hat is an experimental SOS and log parser. GitLab Log Analysis: This experimental project sets up an environment for analyzing GitLab logs using Kibana, Filebeat, Elasticsearch, and Logstash. Upon startup, dashboards are automatically imported and ready for log analysis.|

[DotCom]Kibana provides visualization capabilities on top of the content indexed on an Elasticsearch cluster to find Web based queries. Check the tips and tricks to understand what kind of information you can pull using this tool.
Sentry helps investigate errors across different applications or environments, for example Ruby code related errors. Search on Sentry usually is done using a Correlation ID.
Grafana is used for infrastructure metrics collection for GitLab.com |

[Dedicated]The GitLab Dedicated Logs workflow has information about using Opensearch to view logs. The Observability and Monitoring workflow has information about using Grafana to diagnose performance problems.|

Configuration

[Self-managed] Customers manage the configuration for their self-managed instances. Depending on the deployment method, the configuration will be in `/etc/gitlab/gitlab.rb`, `docker-compose.yml`, `values.yaml` or similar.]

[DotCom] The GitLab.com configurations is managed by the infrastructure team and cannot be modified to meet individual customer's needs.]

[Dedicated] GitLab Dedicated customers can configure their tenants via Switchboard and via the Admin Area. GitLab Support team members can get access Switchboard through Okta by following these steps. Some configuration changes must be done via a Request for Help issue. |

Architecture

[Self-managed] Check out the reference architectures we recommend to our customers and their different variations|

[DotCom] The GitLab.com architecture is managed by the infrastructure team and cannot be modified to meet individual customer's needs.]

[Dedicated] GitLab Dedicated is a fully isolated, single-tenant, SaaS service managed through AWS by the GitLab Dedicated Group. GitLab Dedicated tenants use the GitLab Cloud Native Hybrid reference architectures with high availability enabled. See the list of changes from the reference architectures.]

Auth

[Self-managed] Self Managed Administrators will need to configure the desired auth provider directly in their `gitlab.rb`. Any configuration change on this file will require that you run `gitlab-ctl reconfigure` to successfully apply changes.]

[DotCom] Customers will Configure SAML SSO for Groups directly, by going to Group > Settings > SAML SSO.]

[Dedicated] SAML is supported. Start the investigation by searching the logs. To escalate, raise an issue with the GitLab Dedicated team.]

Geo

[Self-managed] Geo is configured in the `gitlab.rb` file. It requires a premium or higher subscription. The steps must be followed in the order in which they appear.]

[DotCom] Geo is not available for customers to configure on `gitlab.com`.]

[Dedicated] GitLab Geo is supported with some limitations. Raise issue with Dedicated team|

Admin Area

| | |

|:---:| ---- |

[Self-managed] Administrators of self-managed systems will have access to their instance admin.

[DotCom] Only GitLab Team Members can make use of administrative tasks on GitLab.com, therefore access is not granted to customers.

[Dedicated] Customers have instance admins that can access the Admin Area. There is no GitLab Rails console access for customers.

Use the Admin Area docs for reference.

Gitaly

| | |

|:---:| ---- |

[Self-managed] Gitaly configuration is managed through the gitlab.rb file. A customer can configure stand-alone Gitaly or leverage Gitaly Cluster to fulfill its git requirements. Be sure you know which of these types of Gitaly configuration a self-managed customer is using before you begin troubleshooting.

[DotCom] Gitaly is managed by the infrastructure team of GitLab, so customers do not have access to it.

[Dedicated] Gitaly is managed by the Environment Automation SREs. Open an RFH if needed. |

Subscription

| | |

|:---:| ---- |

[Dedicated] GitLab Dedicated customers get Ultimate but please note that some features are not available in GitLab Dedicated.

[DotCom] For folks who make use of GitLab.com, the available plans include Free, Premium and Ultimate.

[Self-managed] For self-managed instances, folks can use either GitLab CE (Community Edition) or GitLab EE (Enterprise Edition) without a license. GitLab EE can be activated with a Premium or an Ultimate license.

Note that GitLab Duo add-ons are handled separately from the subscription.

Runners

| | |

|:---:| ---- |

[Self-managed] Self-managed runners are deployed to customer-owned infrastructure and registered to the self-managed GitLab instance.

[DotCom] GitLab-hosted runners are available to all projects stored on GitLab.com. Self-managed runners can also be registered to GitLab.com. |

[Dedicated] GitLab-hosted runners (beta) can be provisioned for Dedicated customers. Self-managed runners can also be registered to Dedicated environments. |

Rails console

| | |

|:---:| ---- |

|Self-managed| Rails console commands can be run by starting the rails console on any of the rails nodes. |

|DotCom| On GitLab.com, rails console can be accessed by creating an internal request. |

|Dedicated| Rails console is not available for dedicated. In urgent matters please create a request for help with dedicated team. |

Feature flags

| | |

|:---:| ---- |

|Self-managed| Instance administrator can enable feature flags through the rails console. |

|DotCom| On GitLab.com some features flags can be enabled through chatops. |

|Dedicated| Feature Flags are not available for GitLab Dedicated until the feature flag is enabled by default. Read more about how to handle feature flag requests from GitLab Dedicated customers. |

SECTION: support/workflows/sending_notices.md

Overview of the process

At times the Support team will be asked to send notices to GitLab SaaS users or customers to inform them of actions that we have taken (or will be taking) on namespaces or projects under their control.

This workflow describes how to fulfill different types of contact requests and the helpful tools you can use in the process.

Tools and Approvals

| Number of users | Which thing to use | Approvals required | Notifications required |

| --- | --- | --- | --- |

| 1-2 | Manually create a Zendesk ticket | None | None |

| 3+ | Mass Emails through Marketing Department | Director | Support Preparedness issue + FAQ |

- Support team can be asked to contact users during an incident. Such requests are filed by infra team using confidentialincidentdata issue template in production issue tracker. These must be fulfilled by CMOC during the shift.

- As a Stable Counterpart you may choose to manually create tickets for a higher number of users at your discretion. This should be done solely for technical matters ("your usage is causing issues, may we suggest a different approach"), never for marketing reasons ("we have a new way to do xyz and would like you to adopt it").

Avoid sending RED data

In all outbound contact requests explicitly avoid sending RED data. (see: Data Classification Standard). This avoids:

- data transposition errors (accidentally sending another customers data to a customer)
- customer-internal data leaks (person x who received the email was not authorized to know about project y)
- putting customer RED data into systems where it is not allowed (system z is only classified for YELLOW data, but is used to send RED)

If there is any doubt about whether a particular piece of data is appropriate to send, start with asking in supportleadership and escalate to privacy-teamhelp / security as required.

Some examples:

- Don't send the full path (for example, /big-co/sub-proj/super-secret-project), instead send the project/namespace ID and instructions for translating it.

Tips for avoiding RED data in notices

- Make use of the built-in Project and Group redirects when sharing a link to the resource. They will be redirected if the user has appropriate permissions to view the resource.
 - Project example: <https://gitlab.com/-/p/278964> will redirect to <https://gitlab.com/gitlab-org/gitlab>.
 - Group example: <https://gitlab.com/-/g/9970> will redirect to <https://gitlab.com/gitlab-org>.

How to send notices

Most notices should be sent in the form of Zendesk tickets. Always send these tickets to users with Owner level permissions in the namespace or project in question.

Most contact requests will involve contacting all of the owners of only one project or only a few specific users. If you're tasked with contacting the owners of a project and know that there's only one, feel free to look up their email address using your admin account or ChatOps.

However, some contact requests may involve contacting all of the owners of multiple projects. Support Engineers should direct requests for reaching out to multiple owners across multiple projects to do Mass Emails through Marketing Department

Make sure to add an admin note on a user/group we took action on. This will ensure that we can track a block/change reason if a user reaches out to us using a different channel.

Manually create a Zendesk ticket

Warning This often requires the ability to create users in Zendesk, which is only available on specific roles i.e. CMOCs. That access is very risky and should only be used as documented in this section.

Note This only applies to Zendesk Global at this time.

When you need to send an outbound request, it must be done in a very specific manner to ensure it routes properly and the end-user we wish to contact receives the correct notification.

To do this, start by hover over the + Add at the top-left of Zendesk and then click Ticket.

The first thing you need to do is select the requester. To do this, click in the text box below Requester at the top-left of the page and put the email you are looking to reach out to.

The action taken next depend on whether the user in question exists or not:

- If the user exists, it will show said user in a drop-down (clicking said entry in the drop-down will select the user).
- If the user does not exist, no results will be found and you will need to click the + Add user link in the drop-down. Doing so will bring up a modal where you will enter the name and email of the user to create. If you are unsure of the name, it is always safest to use the email itself for both the name and email field. Click the blue Add button to create the user.
- Warning Never create a user in any other fashion. This is the one and only way a user should ever be manually created within our support system by any team other than Support Readiness. Never utilize this method when CC'ing a user.

After doing this, you should add the subject of the ticket. This should always be done via the top-center part of the page.

After filling out the subject, you now need to apply the macro General::Outbound Contact Request. This step is vital and should never be skipped. This macro will ensure the ticket is setup properly for an outbound request. Skipping this step can result in the new ticket not routing properly and encountering an untold number of problems.

After this has been done, you will then enter the description for the ticket. This is the core of the message you are wanting to send to the end-user.

Once this has been completed, the last step is to then finish creating the ticket by submitting it as pending. This is done by clicking the downward arrow at the bottom-right of the page and clicking Submit as Pending.

Mass Emails through Marketing Department

Outside of Zendesk we may be asked to be involved in the process of sending mass notices to users. For larger email campaigns, involve the marketing team:

1. Open an issue in the marketing/marketing-operations issue tracker using the request-operational-email template. You may also need to create a supplementary issue using request-target-list template.
1. Fill the template out in its entirety.
1. Submit the issue and be ready to adjust the subject and/or body of the notice based on feedback.

Note that this process requires a number of approvals from the management, so plan ahead and

make sure that the issue is completely ready for review at least one week before sending out emails.

Tools

The following tools can be used to facilitate different types of contact requests.

Email Grab Script

The Email Grab Script is a Ruby script that will return the primary email addresses of the owners of a group or project. Currently, it can only be used if you possess a GitLab SaaS admin account. It can also be supplied only a list of usernames, in which case it will return the primary email addresses of those users. To use it:

1. Install the required labclient gem with `gem install labclient`.
1. Copy the script to your local machine and give it a name, like `emailgrab.rb`.
1. Replace `YourSuperSweetPAT` with a `readapi` scoped PAT from your GitLab SaaS admin account.

- Note: If a new PAT was created from an admin account, that account will receive a slack message from the SIRTbot app asking if the PAT creation was legitimate. Remember to fill this out for security auditing.

1. Comment out the sections of the script that you will not be using, either groups, users, or projects.
1. Add your data to the section you will be using.
1. Run the script with `ruby emailgrab.rb`.

The results will be created as `namespace-contacts.csv` in the same directory that the script is located in.

SECTION: support/workflows/sentry.md

Overview

Note: Sentry organizes applications into "Sentry Teams". To investigate errors across different applications or environments, our primary teams used are `gitlab` (for the production rails application) and `gitlab-internal` (for non-production environments). By joining said Sentry Team, the application errors should be viewable.

To find the error specifically related to a user on Sentry, try to check for a logged error in at least one of these Sentry projects:

1. `gitlabcom`
1. `gitlabcom-clientside`
1. `customersgitlabcom`

From the main project landing page, you'll see an overview of the application and its performance. On the bottom half of the page you'll see recent issues; click on `Open in issues`

on the right hand side to be taken to all issues on the project.

1. Obtain the full URL the user was visiting when the error occurred along with their user ID, if needed for searching.

1. Log in to Sentry.

1. Enter a query into the search field. For example, after reproducing an error with your admin user. Be aware that usernames are case-sensitive in Sentry.

plaintext

is:unresolved user.username:your-admin

You can also use user.id: or url: for a specific page on GitLab.com.

Use one of the following methods to get a user.id:

- Chatops: Run /chatops run user find <username or email>
- Admin account: Navigate to the admin link in the GitLab User Lookup Zendesk app
- Users API: Search for user using their email or username

At times a search will turn up a Sentry issue that appears to reference the information (user ID, URL, etc...) of another user and not the one that reported the issue. If this happens and you need to create an issue for that specific reporter, simply click the Events tab as seen below to view a list of all users affected by that issue.

You can then click a specific event to view the Sentry issue for that user.

See the Sentry guide and this presentation (GitLab internal only) for more information.

Searching with the username in gitlabcom Sentry project

If you have the username of the user, find the error message for the user in Sentry's gitlabcom project:

- Go to gitlabcom Sentry project
- Use user.username:example (replace example with the actual username from GitLab)
- Open sentry issue · Click on All Events.
- Filter for the specific user again using user.username:example
- Click on any event to see details of the error message

Searching with the user's ID in gitlabcom-clientside Sentry project

If you have the ID of the user, find the error message for the user in Sentry's gitlabcom-clientside project:

- Go to gitlabcom-clientside Sentry project
- Use user.id:userID (replace userID with the actual ID from GitLab)
- Open sentry issue · Click on All Events.

- Filter for the specific user again using user.id:userID
- Click on any event to see details of the error message

Searching customersgitlabcom Sentry project

Use the workflow

to find the error message in Customers Portal Sentry project.

Searching by Correlation ID

In most cases errors in Sentry can be found by searching using user.id:, but this won't always be the case. Sometimes, you may need to search Kibana first to locate the correlation ID that can then be searched for in Sentry.

In the following example, the customer is attempting to change the notification email for one of their groups but receives a 500 error when selecting the desired address from the dropdown list. Searching Sentry for their user.id: turns up nothing, so we need to do the following to find the 500 in Kibana to get the correlation ID that we'll then provide to Sentry.

1. Add positive filters in Kibana for json.username with their GitLab.com username, json.controller for Profiles::NotificationsController, and json.status with 500.
1. Using the left-hand side menu add the json.path, json.controller, json.status, and json.correlationid fields to your search results, which should give you results similar to the following if any errors occurred within your set time range.

1. Pick a correlation ID value and move over to Sentry to search for it using correlationid:, which should give you a result.

SECTION: support/workflows/service_desk_troubleshooting.md

Overview

Users can experience various issues while utilizing Service Desk. This guide walks through troubleshooting when an email does not generate a Service Desk issue.

See also, the Service Desk runbook doc.

Troubleshooting Steps

1. When an issue related to Service Desk is received, verify with the customer these common occurrences are not applicable:

1. Attachments cannot be over 100MiB
1. Emails with Auto-Submitted or X-Autoreply in the header are ignored
1. See the Participants in Service Desk epic for known limitations.

1. If known issues above are not the cause, request the following from the user:

1. The source email (including headers) for an example email sent to the Service Desk which did not create an issue. If possible, request the source email be provided as an .eml file to preserve the email headers.

1. A link to the GitLab.com project which the email was attempting to send to.

1. Did the sender receive a reply (failure)? If yes,

1. Request a screenshot of the failure message.

1. Search Kibana by any of the following:

1. In Rails: Sender's IP address (json.remoteip)

1. In Sidekiq: Service Desk Email (json.toaddress)

1. In Sidekiq: Message ID (json.mailuid)

1. Search the mg.gitlab.com mail logs in Mailgun for suppressions

1. Create an issue in the GitLab issue tracker providing all information found

1. Did the sender receive a reply (failure)? If no,

1. Send a Slack message to production, asking for a check of the spam folder in the incoming mailbox for the Service Desk target email

Email to Service Desk Email Flow

mermaid

graph TD;

subgraph "User Email to Service Desk Issue"

SubGraph1Flow(Email to Service Desk)

SubGraph2Flow(Spam?)

SubGraph3Flow(Mailgun/Support Bot Sends Confirmation to Sender)

Node1[User Email/CRM] --> Node2[Optional: User Alias]

Node2[Optional: User Alias] --> SubGraph1Flow

DoChoice1(Mailgun sends failure notice to sender)

SubGraph1Flow -- Failure --> DoChoice1

SubGraph1Flow -- Success --> SubGraph2Flow

DoChoice3(Send to gmail spam folder and Service Desk email is not processed)

DoChoice4(Send to gmail Inbox and Service Desk email is processed)

SubGraph2Flow -- Yes --> DoChoice3

SubGraph2Flow -- No --> DoChoice4 --> SubGraph3Flow

end

Mermaid source:

text

mermaid

graph TD;

subgraph "User Email to Service Desk Issue"

SubGraph1Flow(Email to Service Desk)

SubGraph2Flow(Spam?)

SubGraph3Flow(Mailgun/Support Bot Sends Confirmation to Sender)

Node1[User Email/CRM] --> Node2[Optional: User Alias]

Node2[Optional: User Alias] --> SubGraph1Flow

DoChoice1(Mailgun sends failure notice to sender)

SubGraph1Flow -- Failure --> DoChoice1

SubGraph1Flow -- Success --> SubGraph2Flow


```
DoChoice3(Send to gmail spam folder and Service Desk email is not processed)
DoChoice4(Send to gmail Inbox and Service Desk email is processed)
SubGraph2Flow -- Yes --> DoChoice3
SubGraph2Flow -- No --> DoChoice4 --> SubGraph3Flow
end
```

SECTION: support/workflows/setting_ticket_priority.md

Definitions

The definitions of support impact are in the Statement of Support.

Setting Ticket Priority

If a customer submits a ticket via the web ticket form, they can choose the starting priority of the ticket - this is based on the Customer Severity field you will see in Zendesk. On ticket creation a trigger sets the main Priority field (not visible to the customer) to match the Customer Severity choice. After ticket creation, changes to Customer Severity do not affect the Priority field or SLAs.

If a customer emails the emergency contact email address, the ticket will get a Priority of 'Urgent'.

Manually setting a ticket's priority in the Priority field (not Customer Priority) in Zendesk will change the overall ticket SLA, for both the first and next replies. This allows support to prioritize tickets and update the urgency during the life of the ticket (for example the initial request may be 'High' priority and then follow up questions may need 'Low' priority.) Customers don't receive automatic notifications upon priority change.

Resetting Ticket Priority

There is a Zendesk Macro General::Changed priority that includes language and links that will be helpful if you need to do this.

Note that the amount of time that passes between our last response and the next response from the customer does not impact the priority of the ticket. For example, if the ticket should have High priority based on the definitions of support impact and the customer only responds to our initial query after a month, the ticket priority should still remain High (assuming the nature of the problem did not change).

Guidelines and Effects

Changing the priority in a ticket will change the SLA, because different priorities have shorter or longer SLAs. Please refer to the guidelines and SLA listed in our statement of support.

SECTION: support/workflows/sidekiq_support.md

Sidekiq support

With our subscription to Sidekiq, GitLab is in principle entitled to one support request per quarter.

To get assistance with issues relating to Sidekiq:

- file issues on <<https://github.com/sidekiq/sidekiq>> if you're seeing behavior that looks like a Sidekiq bug,
- link to that from our GitLab.com issue, and
- email support [at] contribsys.com with a link to the Sidekiq issue.

Of course don't abuse the support email for configuration questions.

SECTION: support/workflows/sla_and_views.md

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

Purpose of this page

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

Assuming a ticket has:

- been triaged, and
- has the correct org associated,

then we need to ensure the ticket:

- has the correct SLA, and
- shows in the correct Zendesk (ZD) view.

Appropriate SLA by plan

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

Tickets should have the appropriate SLA according to the support service levels.

Some values for GitLab Plan for ZD organizations (called Support Level in SFDC) do not receive a SLA if they are linked to a service level. These are not expected to have SLA:

- Community. EDU and OSS program do not include Support, but are in SFDC and synced.
- Expired. Former customer with expired subscription. If you believe this is incorrect, see Handling customers with incorrect expired support.
- Hold. This is rarely used and indicates delayed payment or another issue with the sales process.

Consider contacting the Account Owner (Manager) to clarify the customer's status.

Handling customers with incorrect expired support

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

If you believe a customer marked as Expired support and Former Customer is in fact a current paying customer, follow this process to verify and fix the issue.

1. Open Salesforce and find the customer in question.
1. Click Show feed button at the upper part of the page.

1. Send a message there asking to clarify the customer's status and mention @Sales-Support username in SFDC, they should be able to help with such cases. Make sure that @Sales-Support is converted into clickable username, otherwise Sales Support team will not get your message (see the GIF below):

Example of the message:

plaintext

John Doe (Support Engineer): @Sales-Support, this organization has Support Level set to Expired, and they opened a new ticket. Can you clarify if the support is really expired and if we should decline support for this customer, or this is some kind of error and Support Level should be updated? Customer also provided screenshot of their license and it seems valid.

Prepending your message with your name and role (e.g. John Doe (Support Engineer): helps as everyone in Support is using a shared account so it is not possible to deduce who sent which message.

1. Once you follow the above procedure, mention that in the internal note of the ticket (with SFDC link) so that others can pick up from where you left off.

Note: the same workflow applies if you notice that customer-related

information is not up-to-date on SFDC side and you are not able to update it using our generic Support Admin account.

Fixing tags for tickets with Expired organization

This part should be done in combination with the above section to fix future tickets.

If data in SFDC or CustomersDot show that the customer has a valid subscription you should update the ticket in Zendesk side.

For the specific ticket:

1. Find Tags field where all the tags are listed.
 1. Click x next to formercustomer and expired tags to remove them.
 1. Start typing the appropriate tag name and select the appropriate tag.
- For example, if it is a customer with Bronze subscription, type bronze in Tags field.
1. After making sure that tags are updated correctly, submit the ticket to apply the changes.

Note: When you assign a tag, there is a chance that the ticket will breach immediately. It is not strictly necessary but, if possible, send a public reply before you assign tags to prevent breach. Or, write your public reply and apply the tags, to submit the ticket with both changes at the same time.

Verifying that the ticket now has the proper SLA applied

Now that you've added the appropriate domain, head back to your original ticket and verify that it is associated with the appropriate organization and SLA.

Example: full sequence of actions that should be done to fix incorrectly expired organization

- A ticket comes to Zendesk and you spot that GitLab Plan is shown as Expired.
- First, search for any info about this organization in SFDC and CustomersDot.
- If you found that they have a valid license or subscription i.e. it is not expired, you are likely facing this issue.
- Fix SFDC side first by following the steps above.

Sales Support will usually reply to your message in SFDC, and you will be able to see it in the feed at the organization's page there.

GitLab Plan value on Zendesk side will change from Expired to the valid one after the sync is done.

After that new tickets from this organization will be shown correctly.

- Fix the tags in the ticket you are working using these steps. Note that you do not need to wait for fix on SFDC side, you may fix the tags right away. SLA will appear in the ticket after doing it.

- Update the ticket with the internal comment describing what was done.

No SLA

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

Check the SLA by plan for a list of types that do not receive SLA.

Otherwise, the lack of SLA may be one of the following cases.

Last public reply by agent

Tickets do not have a SLA if the latest public reply is from a Zendesk Agent (Support member). The ticket should be assigned to someone, and there should be nothing to fix.

Customer reply from email address not associated with ticket

When a customer responds to a ticket from an email address that is not included in CC or is not the original requester, the customer's response is recorded as internal.

There is a trigger

which sends an internal note to remind people to add the user to CC and reply, see this issue for more details.

If the email is obviously the original requester, you can merge the users.

Alternatively, add the email of the customer to CC.

Please reply (or let the assignee know), including a note to let the customer that a CC has been added to the ticket. For example:

> We got a note from John with email address <john@domain.org>. From the context it looks like they should be included in the ticket, so for convenience I added them to the CC list. If they shouldn't be included, please let us know so we can remove them.`

After that, the cc'ed user's next replies will not be marked as internal anymore.

Ticket views

> .. Warning

> This page is outdated and should only be referenced for historical purposes. Please do not use any of the processes on this page.

Tickets should show in the appropriate view(s).

Wrong queue

In cases where a ticket is showing in the wrong queue, use the reach out to Customer Support Operations.

Get help

If there's anything that doesn't fit in the relevant workflows, you can investigate and try to fix it.

> Important: Always leave internal comments when doing something on the SFDC side or modifying some tags in Zendesk.

It will help the next engineer to understand what was done in the ticket.

If you have any questions, please ask in the supportoperations Slack channel.

SECTION: support/workflows/support-assisted-customer-requests.md

Overview

At times Support may be called on to assist customers in requests which require taking actions that wouldn't be possible within the normal structure of the application.

- migrating a set of users from one identity provider to another
- bulk removing 2FA from a set of users

The purpose of this workflow is to demonstrate the structure of a runbook that includes the steps required before, during, and after scheduled customer work.

A runbook is an internal document that details how to handle a particular situation in detail. It is highly technical and aimed at the IC doing the action.

Project to store runbooks

The runbooks will be stored in an issue in the runbook project
Due to the nature of the information on the runbook, the contents will be confidential.

Runbook structure

Header

Runbook name		

Runbook description		
DRI		
Schedule at		
Meeting URL		
GitLab Issue		

Runbook name:

Runbook name

A name from the runbook. Please use the format: CUSTOMER - REQUESTED TASK
So for Acme requesting us to enforce 2FA on all of their groups the name would be:
Acme - Enforce 2FA on All Groups

Runbook description

A small summary of the work to be done, including the expected result from the task.

DRI

The owner of the task and main contact for schedule and execution.

Schedule at

The agreed date and time to work with the customer on the task. Include the time zone.

Meeting URL

The zoom meeting URL to use.

GitLab Issue

Link to the GitLab issue created for this request

GitLab - Contacts

Name	Role	Description

A table showing who is attending the call from GitLab and their role during the task.

Customer - Contacts

Name	Role	Description

The customer contact section contains who is joining the call from the customer team and their roles.

This table is optional but a useful element to have to increase the efficiency during the task.

Pre-call checklist

Mark the item if completed otherwise provide details which prevents it.

- [] Inform